VISVESVARAYA TECHNOLOGICAL UNIVERSITY

"JnanaSangama", Belgaum -590014, Karnataka.

LAB REPORT
on

Machine Learning (23CS6PCMAL)

Submitted by

Siddharth Arya (1BM23CS328)

in partial fulfillment for the award of the degree of

BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING

B.M.S. COLLEGE OF ENGINEERING
(Autonomous Institution under VTU)
BENGALURU-560019
Jan-2026 to May-2026

B.M.S. College of Engineering,

Bull Temple Road, Bangalore 560019

(Affiliated To Visvesvaraya Technological University, Belgaum)

Department of Computer Science and Engineering

CERTIFICATE

This is to certify that the Lab work entitled "Machine Learning (23CS6PCMAL)" carried out by **Siddharth Arya (1BM23CS328),** who is bonafide student of **B.M.S. College of Engineering.** It is in partial fulfillment for the award of **Bachelor of Engineering in Computer Science and Engineering** of the Visvesvaraya Technological University, Belgaum. The Lab report has been approved as it satisfies the academic requirements in respect of an Machine Learning (23CS6PCMAL) work prescribed for the said degree.

Sneha S Bagalkot Assistant Professor Department of CSE, BMSCE	Dr. Kavitha Sooda Professor & HOD Department of CSE, BMSCE

Index

Sl. No.	Date	Experiment Title	Page No.
1	30/01/2026	Write a python program to import and export data using Pandas library functions	1
2	06/02/2026	Demonstrate various data pre-processing techniques for a given dataset	8
3	13/02/2026	Implement Linear and Multi-Linear Regression algorithm using appropriate dataset	18
4	27/02/2026	Build Logistic Regression Model for a given dataset	26
5	06/03/2026	Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.	39
6	13/03/2026	Build KNN Classification model for a given dataset.	43
7	08/04/2026	Build Support vector machine model for a given dataset.	49
8	08/04/2026	Implement Random forest ensemble method on a given dataset.	53
9	08/04/2026	Implement Boosting ensemble method on a given dataset.	57
10	10/04/2026	Build k-Means algorithm to cluster a set of data stored in a .CSV file.	62
11	10/04/2026	Implement Dimensionality reduction using Principal Component Analysis (PCA) method.	67

Github Link:
https://github.com/Sid-CS328/6thSem-ML-Lab

Program 1

Write a python program to import and export data using Pandas library functions

Screenshot:

DATE: 30/1/26 PAGE:

Lab 1

Data Pre-Processing

Q Write python code for:

i. To load .csv file into the dataframe

```
import pandas as pd
df = pd.read_csv('housing.csv')
```

ii. To display information of all columns

```
df.info()
```

iii. To display statistical information of all numerical

```
df.describe()
```

iv. To display the count of unique labels for column

```
df['Ocean Proximity'].value_counts()
```

v. To display which attributes have missing values greater than zero

```
missing_values = df.isnull().sum()
missing_values[missing_values > 0]
```

Q In both datasets,

① Which columns have missing values? How do you handle them?

In Diabetes dataset, Urea, Cr, Chol, TG, HDL, LDL, BMI have missing values. While workclass, occupation, native-country have missing values in the Adult dataset.

To handle them, we used mean imputation for numerical values, and mode imputation for categorical data

(2) Which categorical columns did you identify in the dataset? How did you encode them?

In diabetes dataset,

→ Gender, CLASS are categorical columns

→ Label encoding is used

In Adult dataset,

→ workclass, education, marital-status, race, sex are categorical columns

→ Label encoding is used

(3) What is the difference between Min-Max Scaling and Standardization? When would you use one over the other?

Feature	Min-Max	Standardization
Range	[0, 1]	Mean = 0, Std = 1
Formula	$(x - min)/(max - min)$	$(x - mean) / std$
Effect of outliers	Highly sensitive	Less sensitive
Distribution	Preserves original shape	Centers data

★ When to use Min-Max Scaling:

(1) When data has no extreme outliers

(2) When features must lie in a fixed range

(3) Commonly used in KNN, Neural Networks

★ When to use Standardization:

(1) When data contains outliers

(2) When features follow a normal distribution

(3) Commonly used in Linear Regression, SVM, PCA

Code:

```
import pandas as pd

import numpy as np


from sklearn.preprocessing import MinMaxScaler, StandardScaler, LabelEncoder

from sklearn.impute import SimpleImputer


print(diabetes_df.columns)


# Handling Missing Values


import numpy as np

from sklearn.impute import SimpleImputer


columns_with_zero = ['Urea', 'Cr', 'HbA1c', 'Chol', 'TG',

                     'HDL', 'LDL', 'VLDL', 'BMI']


diabetes_df[columns_with_zero] = diabetes_df[columns_with_zero].replace(0, np.nan)


imputer = SimpleImputer(strategy='mean')

diabetes_df[columns_with_zero] = imputer.fit_transform(diabetes_df[columns_with_zero])


# Handle Categorical Data
```

```
from sklearn.preprocessing import LabelEncoder

encoder = LabelEncoder()
diabetes_df['Gender'] = encoder.fit_transform(diabetes_df['Gender'])
diabetes_df['CLASS'] = encoder.fit_transform(diabetes_df['CLASS'])

# Outlier detection & Removal (Using IQR)

Q1 = diabetes_df[numerical_cols].quantile(0.25)
Q3 = diabetes_df[numerical_cols].quantile(0.75)
IQR = Q3 - Q1

diabetes_no_outliers = diabetes_df[
    ~((diabetes_df[numerical_cols] < (Q1 - 1.5 * IQR)) |
      (diabetes_df[numerical_cols] > (Q3 + 1.5 * IQR))).any(axis=1)
]

print("Original shape:", diabetes_df.shape)
print("After outlier removal:", diabetes_no_outliers.shape)

# Min-Max Scaling (Normalisation)

from sklearn.preprocessing import MinMaxScaler
```

```
minmax_scaler = MinMaxScaler()

diabetes_minmax = diabetes_no_outliers.copy()
diabetes_minmax[numerical_cols] = minmax_scaler.fit_transform(
    diabetes_no_outliers[numerical_cols]
)

diabetes_minmax.head()

# Standard Scaling

from sklearn.preprocessing import StandardScaler

standard_scaler = StandardScaler()

diabetes_standard = diabetes_no_outliers.copy()
diabetes_standard[numerical_cols] = standard_scaler.fit_transform(
    diabetes_no_outliers[numerical_cols]
)

diabetes_standard.head()

# Handling Missing Values
```

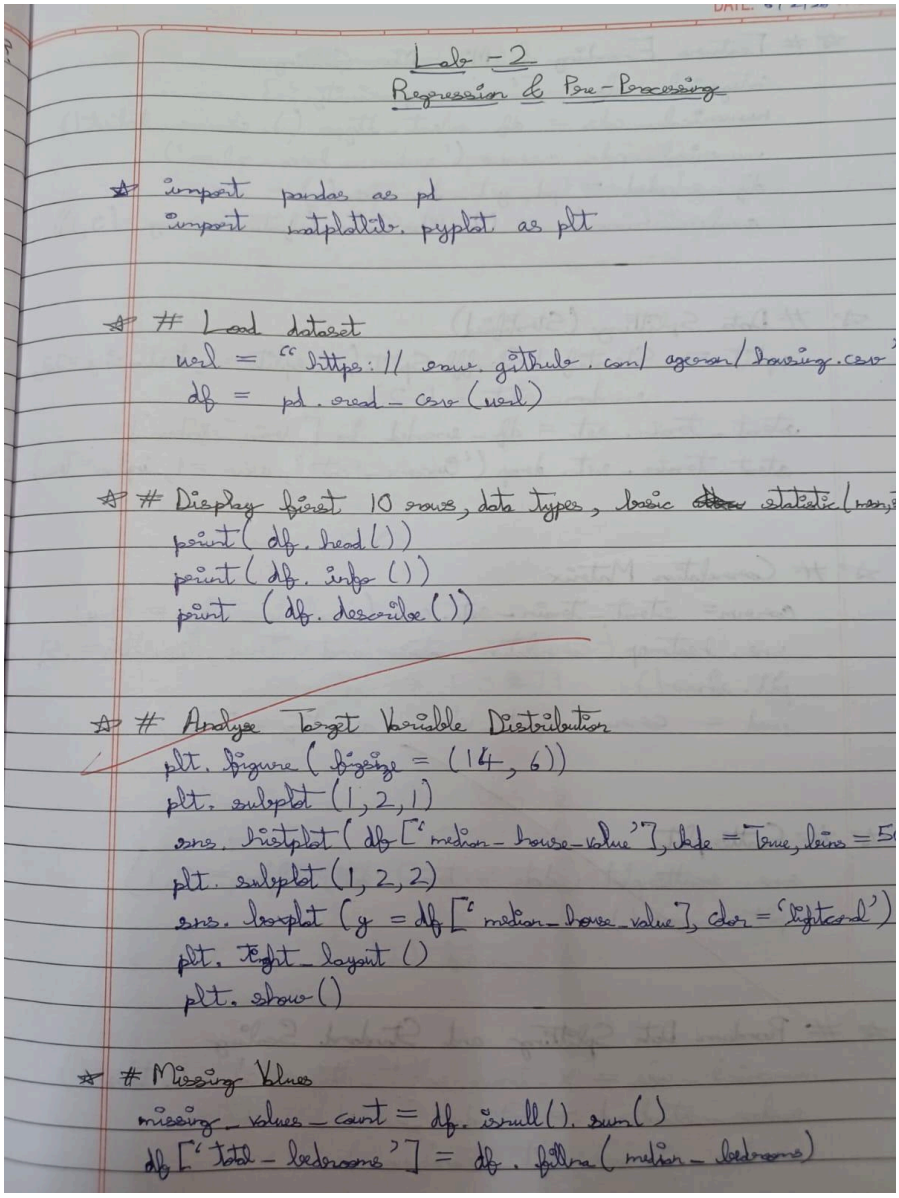
```
adult_df.replace('?', np.nan, inplace=True)

imputer = SimpleImputer(strategy='most_frequent')
adult_df = pd.DataFrame(imputer.fit_transform(adult_df),
                        columns=adult_df.columns)

# Handling Categorical Data

categorical_cols = adult_df.select_dtypes(include='object').columns

encoder = LabelEncoder()
for col in categorical_cols:
    adult_df[col] = encoder.fit_transform(adult_df[col])

# Handling Outliers

numeric_cols = adult_df.select_dtypes(include=np.number).columns

Q1 = adult_df[numeric_cols].quantile(0.25)
Q3 = adult_df[numeric_cols].quantile(0.75)
IQR = Q3 - Q1

adult_no_outliers = adult_df[
    ~((adult_df[numeric_cols] < (Q1 - 1.5 * IQR)) |
```

```
    (adult_df[numeric_cols] > (Q3 + 1.5 * IQR))).any(axis=1)
]

print("Original shape:", adult_df.shape)
print("After outlier removal:", adult_no_outliers.shape)

# Min-Max Scaling (Normalization)

minmax_scaler = MinMaxScaler()

adult_minmax = adult_no_outliers.copy()
adult_minmax[numeric_cols] = minmax_scaler.fit_transform(
    adult_no_outliers[numeric_cols]
)

# Standard Scaling

standard_scaler = StandardScaler()

adult_standard = adult_no_outliers.copy()
adult_standard[numeric_cols] = standard_scaler.fit_transform(
    adult_no_outliers[numeric_cols]
)
```

Program 2

Demonstrate various data pre-processing techniques for a given dataset

Screenshot:



Lab - 2

Regression & Pre-Processing

```
import pandas as pd
import matplotlib.pyplot as plt

# Load dataset
url = "https://raw.github.com/ageron/housing.csv"
df = pd.read_csv(url)

# Display first 10 rows, data types, basic statistic
print(df.head())
print(df.info())
print(df.describe())

# Analyze Target Variable Distribution
plt.figure(figsize = (14, 6))
plt.subplot(1, 2, 1)
sns.histplot(df['median_house_value'], kde = True, bins = 5
plt.subplot(1, 2, 2)
sns.boxplot(y = df['median_house_value'], color = 'lightcoral')
plt.tight_layout()
plt.show()

# Missing Values
missing_values_count = df.isnull().sum()
df['total_bedrooms'] = df.fillna(median_bedrooms)
```

☆ # Feature Encoding & Min-Max Scaling

```
categorical_cols = ['ocean_proximity']
numerical_cols = df.select_dtypes().columns.tolist()
numerical_cols.remove('median_house_values')
df_encoded = pd.get_dummies()
scaler_minmax = MinMaxScaler(feature_range=(0,1))
```

☆ # Data Splitting (Stratified)

```
split = StratifiedShuffleSplit(n_splits=1, test_size=0.2,
        random_state=42)
strat_train_set = df_encoded.loc[train_index]
strat_train_set.drop('income_cat', axis=1, inplace=True)
```

☆ # Correlation Matrix

```
corrm = strat_train_set.corr(numeric_only=True)
sns.heatmap(correlation_matrix, annot=True, linewidths=.5)
plt.show()
med = corrm.sort_values(ascending=False)
```

☆ # Scatterplot

```
sns.scatterplot(data=train_set, alpha=0.1)
plt.show()
```

☆ # Random Data Splitting and Standard Scaling

```
numerical_cols = X_train.select_dtypes.columns.tolist()
scaler_standard = StandardScaler()
scaler_standard.fit_transform()
```

DATE: PAGE:

☆ # Linear Regression Model

```
lin_reg = LinearRegression()
lin_reg.fit()
y_pred = lin_reg.predict(X_test)
rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mae = mean_absolute_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)
```

☆ Output

rmse = 69926.19

mae = 50826.39

r2 score = 0.63

Missing values

~~total_bedrooms 207~~

~~percentage → 1.079~~

median_income 0.684522

ocean_prox_<1H 0.257341

ocean_prox_NEARBAY 0.163476

↑

☆ Most +ve correlated features

median_income

☆ Most -ve correlated features

latitude -0.14416

Code:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns


from sklearn.model_selection import StratifiedShuffleSplit

from sklearn.preprocessing import MinMaxScaler


url = "https://raw.githubusercontent.com/ageron/handson-ml/master/datasets/housing/housing.csv"

housing = pd.read_csv(url)


housing.head()


housing.head(10)

housing.dtypes


housing.describe()


plt.figure(figsize=(12, 5))


# Histogram

plt.subplot(1, 2, 1)

plt.hist(housing["median_house_value"], bins=50)
```

```
plt.title("Histogram of Median House Value")
plt.xlabel("Median House Value")
plt.ylabel("Frequency")

# Box plot
plt.subplot(1, 2, 2)
sns.boxplot(x=housing["median_house_value"])
plt.title("Box Plot of Median House Value")

plt.tight_layout()
plt.show()

missing_values = housing.isnull().sum()
missing_percentage = (missing_values / len(housing)) * 100

missing_data = pd.DataFrame({
    "Missing Values": missing_values,
    "Percentage (%)": missing_percentage
})

missing_data

median_bedrooms = housing["total_bedrooms"].median()
housing["total_bedrooms"] = housing["total_bedrooms"].fillna(median_bedrooms)
```

```
housing_encoded = pd.get_dummies(housing, columns=["ocean_proximity"])
housing_encoded.head()

scaler = MinMaxScaler(feature_range=(0, 1))

numeric_features = housing_encoded.select_dtypes(include=[np.number]).columns

housing_encoded[numeric_features] = scaler.fit_transform(
    housing_encoded[numeric_features]
)

housing_encoded.head()

housing_encoded["income_cat"] = pd.cut(
    housing["median_income"],
    bins=[0., 1.5, 3.0, 4.5, 6., np.inf],
    labels=[1, 2, 3, 4, 5]
)

split = StratifiedShuffleSplit(n_splits=1, test_size=0.2, random_state=42)

for train_index, test_index in split.split(housing_encoded, housing_encoded["income_cat"]):
    strat_train_set = housing_encoded.loc[train_index]
```

```
    strat_test_set = housing_encoded.loc[test_index]

# Drop income_cat after split
for set_ in (strat_train_set, strat_test_set):
    set_.drop("income_cat", axis=1, inplace=True)

corr_matrix = housing.select_dtypes(include=[np.number]).corr()
corr_matrix["median_house_value"].sort_values(ascending=False)

plt.figure(figsize=(12, 10))
sns.heatmap(corr_matrix, cmap="coolwarm", annot=False)
plt.title("Correlation Matrix Heatmap")
plt.show()

plt.figure(figsize=(8, 6))
plt.scatter(housing["median_income"], housing["median_house_value"], alpha=0.3)
plt.xlabel("Median Income")
plt.ylabel("Median House Value")
plt.title("Median Income vs Median House Value")
plt.show()

from sklearn.model_selection import train_test_split

train_set, test_set = train_test_split(
```

```
    housing,
    test_size=0.2,
    random_state=42
)

corr_matrix = housing.select_dtypes(include=[np.number]).corr()

# The top three features most positively correlated with median_house_value
top_positive = corr_matrix["median_house_value"].sort_values(ascending=False)[1:4]
top_positive

# The most negatively correlated feature
most_negative = corr_matrix["median_house_value"].sort_values().head(1)
most_negative

train_set = pd.get_dummies(train_set, columns=["ocean_proximity"])
test_set = pd.get_dummies(test_set, columns=["ocean_proximity"])

train_set, test_set = train_set.align(test_set, join="left", axis=1, fill_value=0)

X_train = train_set.drop("median_house_value", axis=1)
y_train = train_set["median_house_value"]

X_test = test_set.drop("median_house_value", axis=1)
```

```
y_test = test_set["median_house_value"]

from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()

numeric_features = X_train.select_dtypes(include=[np.number]).columns

X_train[numeric_features] = scaler.fit_transform(X_train[numeric_features])
X_test[numeric_features] = scaler.transform(X_test[numeric_features])

from sklearn.linear_model import LinearRegression

lin_reg = LinearRegression()
lin_reg.fit(X_train, y_train)

lin_reg.score(X_train, y_train)

y_pred = lin_reg.predict(X_test)

from sklearn.metrics import mean_squared_error, mean_absolute_error, r2_score

rmse = np.sqrt(mean_squared_error(y_test, y_pred))
mae = mean_absolute_error(y_test, y_pred)
```

```
r2 = r2_score(y_test, y_pred)

print("Model Evaluation Metrics:")
print(f"RMSE : {rmse:.2f}")
print(f"MAE  : {mae:.2f}")
print(f"R²   : {r2:.4f}")

plt.figure(figsize=(8, 6))
plt.scatter(y_test, y_pred, alpha=0.4)
plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    color="red",
    linewidth=2
)

plt.xlabel("Actual Median House Value")
plt.ylabel("Predicted Median House Value")
plt.title("Actual vs Predicted House Values")
plt.show()
```

Program 3

Implement Linear and Multi-Linear Regression algorithm using appropriate dataset

Screenshot:

DATE: 13/2/26

Lab - 3

Linear & Multi - Linear Regression

Q Solve Linear Regression using Matrix approach

x_i (Week)	y_i (Sales in Thousands)
1	2
2	4
3	5
4	9

$$\beta = ((X^T X)^{-1} X^T) Y \qquad X = \begin{bmatrix} 1 & 1 \\ 1 & 2 \\ 1 & 3 \\ 1 & 4 \end{bmatrix}, \quad Y = \begin{bmatrix} 2 \\ 4 \\ 5 \\ 9 \end{bmatrix}$$

$$X^T X = \begin{bmatrix} 4 & 10 \\ 10 & 30 \end{bmatrix}$$

$$(X^T X)^{-1} = \begin{bmatrix} 1.5 & -0.5 \\ -0.5 & 0.2 \end{bmatrix}$$

$$(X^T X)^{-1} X^T = \begin{bmatrix} 1 & 0.5 & 0 & -0.5 \\ -0.3 & -0.1 & 0.1 & 0.3 \end{bmatrix}$$

$$((X^T X)^{-1} X^T) Y = \begin{bmatrix} -0.5 \\ 2.2 \end{bmatrix}$$

$$\therefore \quad y = 2.2x - 0.5$$

★ Canada_per_capita_income dataset

★ Using library

```
import pandas as pd
import numpy as np
from sklearn.linear_model import LinearRegression

model = LinearRegression()
model.fit(X, y)

prediction_2020 = model.predict(pd.DataFrame([[2020]]))
print(prediction_2020[0])
```

★ Outputs

Prediction per capita income in 2020 : 41288.694

DATE: PAGE:

(1) Data pre-processing steps :
For canada-per-capita-income, no steps were required as it contained no missing data & numerical values.
For salary dataset, mean replacement was used for null values.

(2) Visualization
Yes, regression line was plotted.
Strong positive relationship between year and per-capita-inc

(3) Predicted salary
92268.0722

(4) Yes, we encoded categorical variables
One-Hot encoding was used
because regression model cannot process input

13/2/26

Code:

```
import pandas as pd


df = pd.read_csv("canada_per_capita_income.csv")

df.head()


# Using Library

# Expected Prediction: ≈ $41288



import pandas as pd

import numpy as np

from sklearn.linear_model import LinearRegression



df = pd.read_csv("canada_per_capita_income.csv")



X = df[['year']]

y = df['per capita income (US$)']



model = LinearRegression()

model.fit(X, y)
```

```
prediction_2020 = model.predict(pd.DataFrame([[2020]], columns=['year']))

print("Predicted per capita income in 2020:", prediction_2020[0])

#Without using libraries

import numpy as np
import pandas as pd

df = pd.read_csv("canada_per_capita_income.csv")

X = df['year'].values
y = df['per capita income (US$)'].values


X_b = np.c_[np.ones(len(X)), X]


beta = np.linalg.inv(X_b.T.dot(X_b)).dot(X_b.T).dot(y)

intercept = beta[0]
slope = beta[1]
```

```
prediction_2020 = intercept + slope * 2020

print("Intercept:", intercept)
print("Slope:", slope)
print("Predicted income in 2020:", prediction_2020)

# Salary dataset

df2 = pd.read_csv("salary.csv")
df2.head()

# Using library
# Expected Prediction: ≈ $139000


import pandas as pd
from sklearn.linear_model import LinearRegression

df = pd.read_csv("salary.csv")

df = df.dropna()
```

```
X = df[['YearsExperience']]
y = df['Salary']

model = LinearRegression()
model.fit(X, y)

prediction = model.predict(pd.DataFrame([[12]], columns=['YearsExperience']))
print("Predicted salary for 12 years experience:", prediction[0])

# Without library

import numpy as np
import pandas as pd

df = pd.read_csv("salary.csv")
df = df.dropna()

X = df['YearsExperience'].values
y = df['Salary'].values


X_b = np.c_[np.ones(len(X)), X]
```

```
beta = np.linalg.inv(X_b.T.dot(X_b)).dot(X_b.T).dot(y)


intercept = beta[0]

slope = beta[1]



prediction = intercept + slope * 12


print("Intercept:", intercept)

print("Slope:", slope)

print("Predicted Salary:", prediction)
```

Program 4

Build Logistic Regression Model for a given dataset

Screenshot:

DATE: 27/2/26 PAGE:

Lab - 4

Logistic Regression

Q Consider a binary classification problem, where $a_0 = -5, a_1 = 0.8$

→ The logistic regression model is:

$$P = \frac{1}{1 + e^{-(a_0 + a_1 x)}}$$

$$= \frac{1}{1 + e^{-(-5 + 0.8x)}}$$

$$P(\text{pass} \mid x) = \frac{1}{1 + e^{5 - 0.8x}}$$

→ Probability for a student who studies for 7 hours

$$z = -5 + 0.8(7) = 0.6$$

$$P = \frac{1}{1 + e^{-0.6}}$$

$$P = 0.646$$

→ If threshold is 0.5, then

$$0.646 > 0.5$$

∴ Predicted class = Pass

DATE: PAGE:

Q Consider $z = [2, 1, 0]$ for three classes. Apply SoftMax function to find the probability values of three classes.

$$P(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}}$$

$e^2 = 7.389$

$e^1 = 2.718$

$e^0 = 1$

Sum $= 7.389 + 2.718 + 1 = 11.107$

$$P_1 = \frac{7.389}{11.107} = 0.665$$

$$P_2 = \frac{2.718}{11.107} = 0.245$$

$$P_3 = \frac{1}{11.107} = 0.090$$

Find SoftMax probabilities $\doteq [0.665, 0.245, 0.090]$

1. (i) Variables with direct and clear impact on employee retention are as follows:

(1) satisfaction_level : lower means more likely to leave
(2) salary : lower salary indicates more likely to leave
(3) number_projects : employees with very few or too many projects may leave

These variables showed a clear numerical difference between employees who left vs stayed in EDA.

(ii) The accuracy was 78.6 %
This accuracy is good enough considering the dataset is slightly imbalanced.

2. (i) Data preprocessing steps:

(1) Converted all categorical features into numeric form
(2) Applied feature scaling using 'StandardScaler' to improve optimization for logistic Regression.

(ii) No missing or inconsistent values so no imputation needed.

(iii) Most diagonal values (predicted = actual) are high.
This means the model predicts most animals correctly.

(iv) Classes with fewer examples in the dataset (rare animals) were more frequently misclassified.
This happened because the animals share similar features with other classes.

Code:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns


from sklearn.model_selection import train_test_split

from sklearn.linear_model import LogisticRegression

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report


from sklearn.preprocessing import StandardScaler


from google.colab import files

uploaded = files.upload()


df = pd.read_csv("HR_comma_sep.csv")

df.head()


df.info()

df.describe()

df.isnull().sum()


df['left'].value_counts()

df.groupby('left').mean(numeric_only=True)
```

```
pd.crosstab(df.salary, df.left).plot(kind='bar')

plt.title("Employee Retention by Salary")
plt.xlabel("Salary Level")
plt.ylabel("Number of Employees")
plt.legend(["Stayed", "Left"])
plt.show()

pd.crosstab(df.Department, df.left).plot(kind='bar', figsize=(10,5))

plt.title("Employee Retention by Department")
plt.xlabel("Department")
plt.ylabel("Number of Employees")
plt.legend(["Stayed", "Left"])
plt.xticks(rotation=45)
plt.show()

X = df.drop('left', axis=1)

X = pd.get_dummies(X, drop_first=True)

y = df['left']
```

```
scaler = StandardScaler()
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LogisticRegression(max_iter=2000)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print("Model Accuracy:", accuracy)

cm = confusion_matrix(y_test, y_pred)

print(cm)

plt.figure(figsize=(5,4))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.title("Confusion Matrix")
plt.show()
```

```
print(classification_report(y_test, y_pred))

# MultiClass

zoo = pd.read_csv("zoo-data.csv")
class_type = pd.read_csv("zoo-class-type.csv")

zoo.head()

zoo = zoo.drop("animal_name", axis=1)
zoo.isnull().sum()

X = zoo.drop("class_type", axis=1)
y = zoo["class_type"]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.3, random_state=42)

scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

model = LogisticRegression(max_iter=3000, multi_class='multinomial')
```

```
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

accuracy = accuracy_score(y_test, y_pred)
print("Model Accuracy:", accuracy)

print(classification_report(y_test, y_pred))

cm = confusion_matrix(y_test, y_pred)

plt.figure(figsize=(8,6))
sns.heatmap(cm, annot=True, fmt='d', cmap='Blues')

plt.xlabel("Predicted Class")
plt.ylabel("Actual Class")
plt.title("Confusion Matrix")
plt.show()

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
```

```
df = pd.read_csv("HR_comma_sep.csv")

X = df[['satisfaction_level']].values

y = df['left'].values.reshape(-1,1)


X_mean = X.mean()

X_std = X.std()

X_scaled = (X - X_mean)/X_std


X_scaled = np.hstack([np.ones((X_scaled.shape[0],1)), X_scaled])


def sigmoid(z):

    return 1 / (1 + np.exp(-z))


theta = np.zeros((X_scaled.shape[1],1))

alpha = 0.1

iterations = 5000

m = X_scaled.shape[0]


for i in range(iterations):

    z = np.dot(X_scaled, theta)

    h = sigmoid(z)

    gradient = np.dot(X_scaled.T, (h - y)) / m

    theta -= alpha * gradient
```

```
y_prob = sigmoid(np.dot(X_scaled, theta))
y_pred = (y_prob >= 0.5).astype(int)

accuracy = np.mean(y_pred == y)
print("Binary Logistic Regression Accuracy:", accuracy)

x_vals = np.linspace(X_scaled[:,1].min(), X_scaled[:,1].max(), 300)
x_plot = np.c_[np.ones((300,1)), x_vals.reshape(-1,1)]
y_curve = sigmoid(np.dot(x_plot, theta))

plt.figure(figsize=(8,6))
plt.scatter(X_scaled[:,1], y, alpha=0.3, label="Actual Data")
plt.plot(x_vals, y_curve, color='red', linewidth=3, label="Logistic Curve")
plt.xlabel("Satisfaction Level (scaled)")
plt.ylabel("Probability of Leaving")
plt.title("Binary Logistic Regression Fit")
plt.legend()
plt.grid(True)
plt.show()

def confusion_matrix_manual(y_true, y_pred):
    tp = np.sum((y_true==1) & (y_pred==1))
    tn = np.sum((y_true==0) & (y_pred==0))
    fp = np.sum((y_true==0) & (y_pred==1))
```

```
    fn = np.sum((y_true==1) & (y_pred==0))
    return np.array([[tn, fp],[fn, tp]])

cm = confusion_matrix_manual(y, y_pred)
print("Confusion Matrix:\n", cm)

plt.figure(figsize=(5,4))
plt.imshow(cm, cmap='Blues')
plt.colorbar()
plt.title("Confusion Matrix")
plt.xticks([0,1], ["Stayed","Left"])
plt.yticks([0,1], ["Stayed","Left"])
for i in range(2):
    for j in range(2):
        plt.text(j,i,cm[i,j], ha='center', va='center', color='red', fontsize=14)
plt.show()

zoo = pd.read_csv("zoo-data.csv")
X = zoo.drop(['animal_name','class_type'], axis=1).values
y = zoo['class_type'].values

classes = np.unique(y)
num_classes = len(classes)
m, n = X.shape
```

```
X = np.hstack([np.ones((m,1)), X])


def sigmoid(z):
    return 1/(1+np.exp(-z))


alpha = 0.1
iterations = 1000


theta = np.zeros((n+1, num_classes))


Y = np.zeros((m, num_classes))
for i, cls in enumerate(classes):
    Y[:,i] = (y==cls).astype(int)


for k in range(num_classes):
    th = np.zeros((n+1,1))
    yk = Y[:,k].reshape(-1,1)
    for i in range(iterations):
        z = np.dot(X, th)
        h = sigmoid(z)
        grad = np.dot(X.T, (h - yk)) / m
        th -= alpha * grad
    theta[:,k] = th.flatten()
```

```
prob = sigmoid(np.dot(X, theta))
y_pred = np.argmax(prob, axis=1) + 1
accuracy = np.mean(y_pred == y)
print("Multiclass Logistic Regression Accuracy:", accuracy)

cm = np.zeros((num_classes, num_classes), dtype=int)
for i in range(m):
    cm[y[i]-1, y_pred[i]-1] += 1

print("Confusion Matrix:\n", cm)
plt.figure(figsize=(8,6))
plt.imshow(cm, cmap='Blues')
plt.colorbar()
plt.title("Multiclass Confusion Matrix")
plt.xlabel("Predicted")
plt.ylabel("Actual")
plt.xticks(range(num_classes), classes)
plt.yticks(range(num_classes), classes)
for i in range(num_classes):
    for j in range(num_classes):
        plt.text(j,i,cm[i,j], ha='center', va='center', color='red')
plt.show()
```

Program 5

Use an appropriate data set for building the decision tree (ID3) and apply this knowledge to classify a new sample.

Screenshot:

Lab - 6

Decision Tree

Q Consider the following dataset, calculate Entropy & Information Gain w.r.t target variable "Classification". Identify whether the splitting node should be a2 or a3 attribute.

Instance	a2	a3	Classification
1	Hot	High	No
2	Hot	High	No
6	Cool	High	No
7	Hot	High	No
8	Hot	Normal	Yes

* For the overall dataset,

count of 'Yes' = 1

count of 'No' = 4

Total instances = 5

$$\text{Entropy}(S) = -p_+ \log_2(p_+) - p_- \log_2(p_-)$$
$$= -(0.2 \log_2(0.2)) - 0.8 \log_2(0.8)$$
$$\text{Entropy}(S) = 0.7219$$

* For attribute a2

Values = {'Hot', 'Cool'}

→ For a2 = Hot

$p_+ = 0.25$, $p_- = 0.75$

$$\text{Entropy}(\text{Hot}) = -(0.25 \log_2(0.25)) - 0.75 \log_2(0.$$
$$= 0.8113$$

→ For a2 = Cool

$p_+ = 0$, $p_- = 1$, $\therefore \text{Entropy}(\text{Cool}) = 0$

DATE: PAGE:

Information Gain of a2 = Entropy (S) − $\left(\frac{|Hot|}{|S|}\text{Entropy}(S) + \frac{|Cold|}{|S|}\right.$

$= 0.7219 - \left(\frac{4}{5} \times 0.8113 + \frac{1}{5} \times 0\right)$

$= 0.7219 - 0.649$

$= 0.0729$

★ For attribute a3

values = {'High', 'Normal'}

→ For a3 = High

$p_+ = 0$

$p_- = 4$

∴ Entropy (High) = 0

→ For a3 = Normal

$p_+ = 1, p_- = 0$

Entropy (Normal) = 0

Information Gain (I.G) of a3 = ~~Entropy (S) −~~ $0.7219 - \left(\frac{4}{5} \times 0 + \frac{1}{5} \times 0\right)$

$= 0.7219$

★ Since Information Gain (a3) > Information Gain (a2),

i.e., $0.7219 > 0.0729$,

the splitting node should be a3 attribute.

Code:

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.tree import DecisionTreeClassifier

from sklearn.metrics import accuracy_score, confusion_matrix

from sklearn.preprocessing import LabelEncoder


iris = pd.read_csv("iris.csv")


X = iris.iloc[:, :-1]

y = iris.iloc[:, -1]


X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)


model = DecisionTreeClassifier()


model.fit(X_train, y_train)


y_pred = model.predict(X_test)


print("IRIS DATASET")

print("Accuracy:", accuracy_score(y_test, y_pred))

print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

```
drug = pd.read_csv("drug.csv")

le = LabelEncoder()

for column in drug.columns:
    if drug[column].dtype == 'object':
        drug[column] = le.fit_transform(drug[column])

X = drug.iloc[:, :-1]
y = drug.iloc[:, -1]

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

model = DecisionTreeClassifier()

model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("\nDRUG DATASET")
print("Accuracy:", accuracy_score(y_test, y_pred))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred))
```

Program 6

Build KNN Classification model for a given dataset.

Screenshot:

DATE: 6/3/26 PAGE:

Lab - 5

KNN Classification Model

Q Consider a dataset with K = 3 & test data (X, 35, 100) as (Person, Age, Salary K). Solve using KNN classifier model & predict the target.

Using Euclidean distance formula:

$$\text{Distance} = \sqrt{(Age_2 - Age_1)^2 + (Salary\,K_2 - Salary\,K_1)^2}$$

Test point = (35, 100)

Distance to A (18, 50) = $\sqrt{(35-18)^2 + (100-50)^2}$ = 52.81

Distance to B (23, 55) = $\sqrt{(35-23)^2 + (100-55)^2}$ = 46.57

Distance to C (24, 70) = $\sqrt{(35-24)^2 + (100-70)^2}$ = 31.95

Distance to D (41, 60) = $\sqrt{(35-41)^2 + (100-60)^2}$ = 40.45

Distance to E (43, 70) = $\sqrt{(35-43)^2 + (100-70)^2}$ = 31.05

Distance to F (38, 40) = $\sqrt{(35-38)^2 + (100-40)^2}$ = 60.07

Sorting the distances in ascending order & choosing the 3 nearest neighbours gives

E → Y

C → N

D → Y

Since the majority for the target variable is Y, the predicted target for Person X is Y.

★ Output

★ For iris & diabetes dataset :

Accuracy score for iris : 1.00

Optimal K value : 7

Accuracy Score for diabetes : 0.7078

★ For heart.csv dataset :

Optimal K value : 7

with accuracy of 0.9180

Q For Iris dataset :

The value of K affects the performance of the KNN model significantly.

→ Small K (eg 1) : Model may overfit

→ Large K : Model may underfit

Q For Diabetes dataset :

Without feature scaling, the KNN algorithm will essentially ignore the Pedigree function because of massive differences between the features.

The 2 most common ways to perform it are Standardization (Z-score scaling) and Normalization (Min-Max Scaling).

Code:

```
import pandas as pd

import numpy as np

import matplotlib.pyplot as plt

import seaborn as sns

from sklearn.model_selection import train_test_split, GridSearchCV

from sklearn.neighbors import KNeighborsClassifier

from sklearn.preprocessing import StandardScaler

from sklearn.metrics import accuracy_score, confusion_matrix, classification_report


def plot_confusion_matrix(cm, classes, title, cmap):

    plt.figure(figsize=(6, 4))

    sns.heatmap(cm, annot=True, fmt='d', cmap=cmap,

                xticklabels=classes, yticklabels=classes)

    plt.title(title)

    plt.xlabel('Predicted Label')

    plt.ylabel('True Label')

    plt.show()


print("========== TASK 1: IRIS DATASET ==========")

try:

    iris_df = pd.read_csv("iris.csv")

    X_iris = iris_df.iloc[:, :-1].values

    y_iris = iris_df.iloc[:, -1].values
```

```
    X_train_i, X_test_i, y_train_i, y_test_i = train_test_split(X_iris, y_iris, test_size=0.2,
random_state=42)


    knn_iris = KNeighborsClassifier(n_neighbors=3)

    knn_iris.fit(X_train_i, y_train_i)


    y_pred_i = knn_iris.predict(X_test_i)

    acc_iris = accuracy_score(y_test_i, y_pred_i)

    cm_iris = confusion_matrix(y_test_i, y_pred_i)


    print(f"Accuracy Score: {acc_iris:.4f}\n")

    print("Confusion Matrix:\n", cm_iris, "\n")

    print("Classification Report:\n", classification_report(y_test_i, y_pred_i))


    iris_classes = np.unique(y_iris)

    plot_confusion_matrix(cm_iris, iris_classes, 'Iris Dataset Confusion Matrix\n(K=3)', 'Greens')


except FileNotFoundError:

    print("Error: 'iris.csv' not found. Please ensure the file is in the same directory.")


print("\n")
```

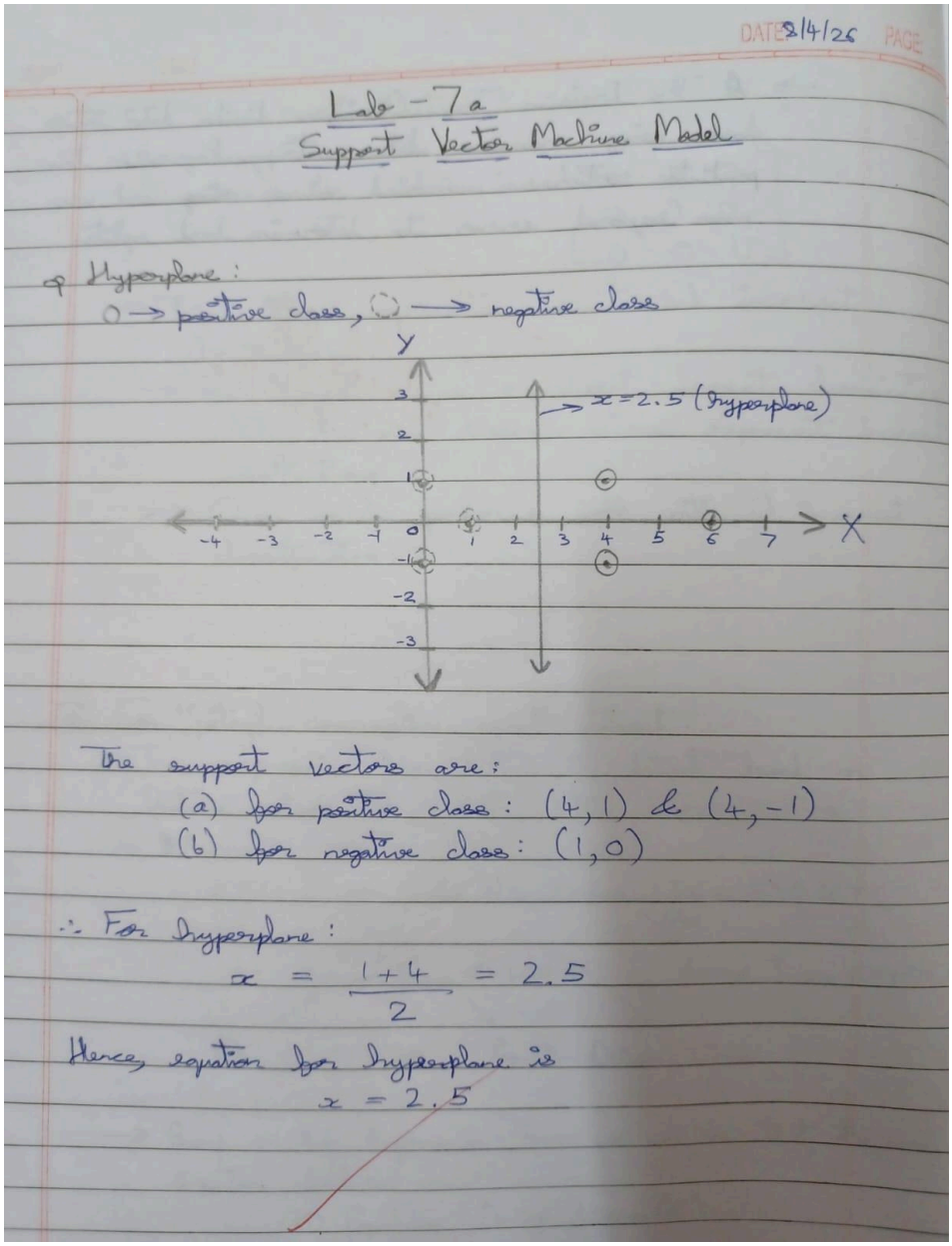
```
print("========== TASK 2: DIABETES DATASET ==========")

try:

    diabetes_df = pd.read_csv("diabetes.csv")

    X_diab = diabetes_df.iloc[:, :-1].values

    y_diab = diabetes_df.iloc[:, -1].values


    X_train_d, X_test_d, y_train_d, y_test_d = train_test_split(X_diab, y_diab, test_size=0.2,
random_state=42)


    scaler = StandardScaler()

    X_train_scaled = scaler.fit_transform(X_train_d)

    X_test_scaled = scaler.transform(X_test_d)


    param_grid = {'n_neighbors': np.arange(1, 31)}

    knn_gscv = GridSearchCV(KNeighborsClassifier(), param_grid, cv=5)

    knn_gscv.fit(X_train_scaled, y_train_d)


    best_k = knn_gscv.best_params_['n_neighbors']

    print(f"Optimal K value found via Grid Search: {best_k}")


    best_knn = KNeighborsClassifier(n_neighbors=best_k)

    best_knn.fit(X_train_scaled, y_train_d)


    y_pred_d = best_knn.predict(X_test_scaled)

    acc_diab = accuracy_score(y_test_d, y_pred_d)
```

```
    cm_diab = confusion_matrix(y_test_d, y_pred_d)


    print(f"Accuracy Score: {acc_diab:.4f}\n")

    print("Confusion Matrix:\n", cm_diab)


    diab_classes = ['Negative (0)', 'Positive (1)']

    plot_confusion_matrix(cm_diab, diab_classes, f'Diabetes Dataset Confusion Matrix\n(Optimized
K={best_k})', 'Blues')


except FileNotFoundError:

    print("Error: 'diabetes.csv' not found. Please ensure it is in the same directory.")
```

Program 7

Build Support vector machine model for a given dataset.

Screenshot:



DATE 2/4/25 PAGE

Lab - 7a

Support Vector Machine Model

Hyperplane:

O → positive class, ⊙ → negative class

The support vectors are:

(a) for positive class: (4, 1) & (4, -1)

(b) for negative class: (1, 0)

∴ For hyperplane:

$$x = \frac{1+4}{2} = 2.5$$

Hence, equation for hyperplane is

$$x = 2.5$$

DATE: PAGE:

★ Output

★ IRIS dataset

Linear Kernel Accuracy : 0.9667

Confusion matrix : $\begin{bmatrix} 10 & 0 & 0 \\ 0 & 8 & 1 \\ 0 & 0 & 11 \end{bmatrix}$

RBF Kernel Accuracy : 1.0

Confusion matrix : $\begin{bmatrix} 10 & 0 & 0 \\ 0 & 9 & 0 \\ 0 & 0 & 11 \end{bmatrix}$

★ Letter Recognition dataset

Accuracy : 0.95

AUC score : 0.999

Q ① For iris dataset,

→ Accuracy score for :

Linear Kernel = 0.9667

RBF = 1.0

→ RBF kernel gave better performance. This is because the IRIS dataset is almost linearly separable even though some overlap exists. This gives the RBL (Radial Basis Function) kernel a slight advantage as it can handle non-linear boundaries.

Code:

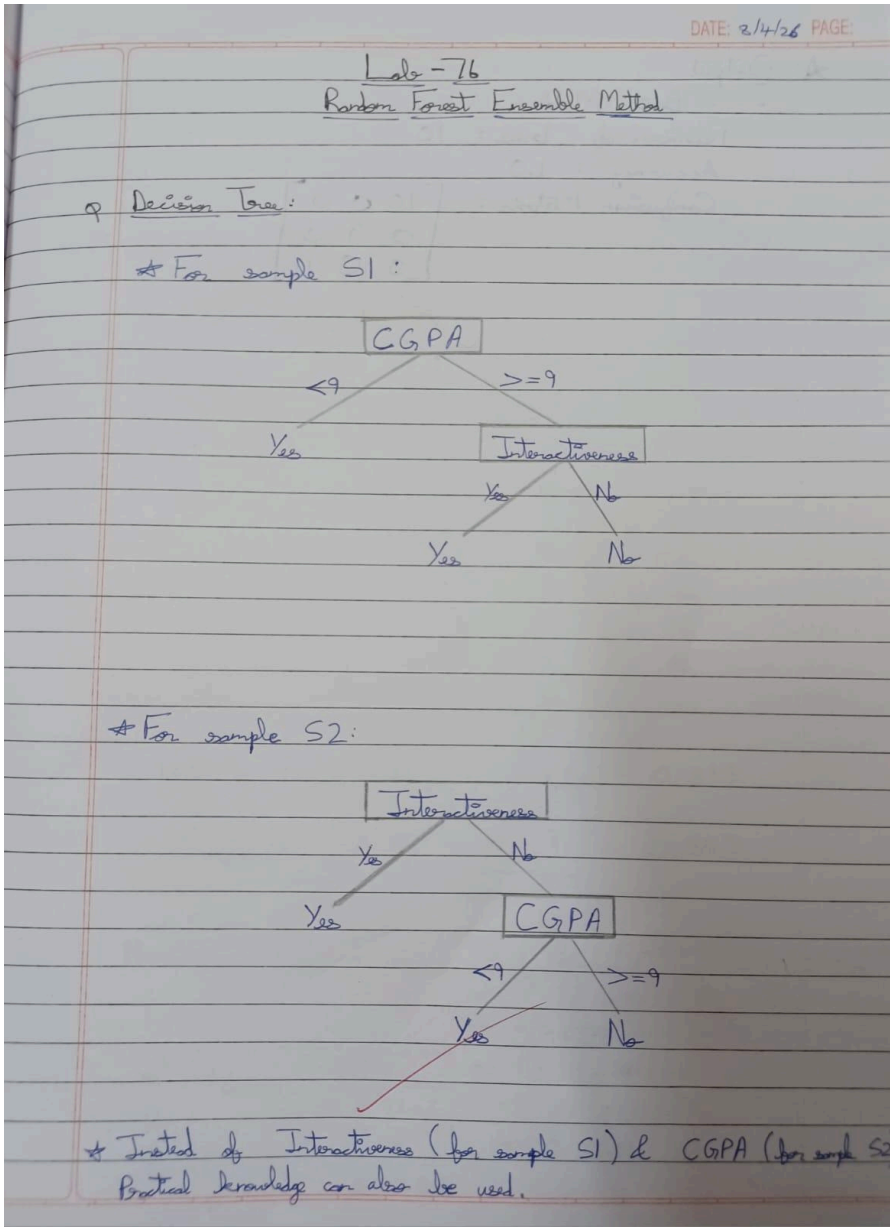
```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.svm import SVC

from sklearn.metrics import accuracy_score, confusion_matrix

from sklearn.preprocessing import StandardScaler


data = pd.read_csv("iris.csv")


X = data.iloc[:, :-1]

y = data.iloc[:, -1]


X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)


scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)

X_test = scaler.transform(X_test)


linear_model = SVC(kernel='linear')

linear_model.fit(X_train, y_train)


y_pred_linear = linear_model.predict(X_test)
```

```
print("Linear Kernel Accuracy:", accuracy_score(y_test, y_pred_linear))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_linear))

rbf_model = SVC(kernel='rbf')
rbf_model.fit(X_train, y_train)

y_pred_rbf = rbf_model.predict(X_test)

print("\nRBF Kernel Accuracy:", accuracy_score(y_test, y_pred_rbf))
print("Confusion Matrix:\n", confusion_matrix(y_test, y_pred_rbf))
```

Program 8

Implement Random forest ensemble method on a given dataset.

Screenshot:



DATE: 2/4/26 PAGE:

Lab - 7b

Random Forest Ensemble Method

Q Decision Tree:

* For sample S1:

CGPA
<9 → Yes
>=9 → Interactiveness
Yes → Yes
No → No

* For sample S2:

Interactiveness
Yes → Yes
No → CGPA
<9 → Yes
>=9 → No

* Instead of Interactiveness (for sample S1) & CGPA (for sample S2) Practical knowledge can also be used.

DATE:

★ Output

---- Best Model ----

Number of Trees : 10

Accuracy : 1.0

Confusion Matrix :

$$\begin{bmatrix} 10 & 0 & 0 \\ 0 & 9 & 0 \\ 0 & 0 & 11 \end{bmatrix}$$

Code:

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score


data = pd.read_csv("iris.csv")


X = data.iloc[:, :-1]

y = data.iloc[:, -1]


X_train, X_test, y_train, y_test = train_test_split(

    X, y, test_size=0.2, random_state=42

)



rf_default = RandomForestClassifier(n_estimators=10, random_state=42)

rf_default.fit(X_train, y_train)


y_pred_default = rf_default.predict(X_test)

default_acc = accuracy_score(y_test, y_pred_default)


print("Accuracy with default (10 trees):", round(default_acc, 3))
```

```
tree_range = [10, 20, 50, 100, 150, 200]
best_score = 0
best_trees = 0

print("\nTuning Results:")

for n in tree_range:
    rf = RandomForestClassifier(n_estimators=n, random_state=42)
    rf.fit(X_train, y_train)

    y_pred = rf.predict(X_test)
    acc = accuracy_score(y_test, y_pred)

    print(f"Trees: {n}, Accuracy: {round(acc, 3)}")

    if acc > best_score:
        best_score = acc
        best_trees = n

print("\nBest Accuracy:", round(best_score, 3))
print("Best number of trees:", best_trees)
```

Program 9

Implement Boosting ensemble method on a given dataset.

Screenshot:

DATE: 8/4/26 PAGE:

Lab - 8
Ada Boost

Q Decision Stump calculation steps:

① Initial weight $= w_i = \frac{1}{N} = \frac{1}{6}$

② Decision Stump for CGPA:

CGPA	Predicted Job Offer	Actual Job Offer	Weight
>=9	Yes	Yes	$\frac{1}{6}$
<9	No	Yes	$\frac{1}{6}$
>=9	Yes	No	$\frac{1}{6}$
<9	No	No	$\frac{1}{6}$
>=9	Yes	Yes	$\frac{1}{6}$
>=9	Yes	Yes	$\frac{1}{6}$

Weighted Error $= \varepsilon_{CGPA} = 2 \times \frac{1}{6} = 0.333$

$$\alpha_{CGPA} = \frac{1}{2} \frac{\ln(1-\varepsilon_{CGPA})}{\varepsilon_{CGPA}} = 0.347$$

Normalizing factor $= Z_{CGPA} = \frac{1}{6}(4)(e^{-0.347}) + \frac{1}{6}(2)(e$

$= 0.9428$

$$wt(d_j)_{i+1}^{+} = \frac{\left(\frac{1}{6}\right) e^{-0.347}}{0.9428} = 0.1249$$

$$wt(d_j)_{i+1}^{-} = \frac{\left(\frac{1}{6}\right) e^{0.347}}{0.9428} = 0.2501$$

DATE:

★ Output

#Best Model#

Accuracy :0.834

Number of Trees : 500

Confusion Matrix : $\begin{bmatrix} 7091 & 323 \\ 1299 & 1056 \end{bmatrix}$

Code:

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.ensemble import AdaBoostClassifier

from sklearn.metrics import accuracy_score

from sklearn.preprocessing import LabelEncoder


try:

    df = pd.read_csv("income.csv")

except FileNotFoundError:

    print("Error: 'income.csv' not found. Please ensure the file is in the same directory.")



target_col = df.columns[-1]


X = df.drop(columns=[target_col])

y = df[target_col]


X = pd.get_dummies(X, drop_first=True)


le = LabelEncoder()

y = le.fit_transform(y)


X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
```

```
print("--- Initial Model ---")
ada_initial = AdaBoostClassifier(n_estimators=10, random_state=42)
ada_initial.fit(X_train, y_train)

y_pred_initial = ada_initial.predict(X_test)
score_10 = accuracy_score(y_test, y_pred_initial)

print(f"Accuracy with n_estimators=10: {score_10 * 100:.2f}%\n")


print("--- Fine-tuning Model ---")

n_estimators_list = [10, 50, 100, 150, 200, 300, 500]

best_score = 0
best_n_trees = 0

for n_trees in n_estimators_list:
    clf = AdaBoostClassifier(n_estimators=n_trees, random_state=42)
    clf.fit(X_train, y_train)

    y_pred = clf.predict(X_test)
    current_score = accuracy_score(y_test, y_pred)
```

```
    print(f"Accuracy with {n_trees:3} trees: {current_score * 100:.2f}%")

    if current_score > best_score:
        best_score = current_score
        best_n_trees = n_trees

print("\n--- Final Result ---")
print(f"The best prediction score is {best_score * 100:.2f}% achieved using {best_n_trees} trees.")
```

Program 10

Build k-Means algorithm to cluster a set of data stored in a .CSV file.

Screenshot:

DATE: 10/4/26 PAGE:

Lab - 9

K - Means Algorithm

a Compute Clusters:

Initial Cluster Centers:

$C_1 = (1.0, 1.0)$

$C_2 = (5.0, 7.0)$

Iteration 1:

Record	Point (x, y)	D^2 to C_1	D^2 to C_2	Cluster Assignment
R_1	(1, 1)	0	52	Cluster 1
R_2	(1.5, 2)	1.25	37.25	Cluster 1
R_3	(3, 4)	13	13	Cluster 1
R_4	(5, 7)	52	0	Cluster 2
R_5	(3.5, 5)	22.25	6.25	Cluster 2
R_6	(4.5, 5)	28.25	4.25	Cluster 2
R_7	(3.5, 4.5)	18.5	8.5	Cluster 2

Cluster 1 (K_1): $\{R_1, R_2, R_3\}$

Cluster 2 (K_2): $\{R_4, R_5, R_6, R_7\}$

New C_1: $\left(\frac{1+1.5+3}{3}, \frac{1+2+4}{3}\right) = (1.833, 2.333)$

New C_2: $\left(\frac{5+3.5+4.5+3.5}{4}, \frac{7+5+5+4.5}{4}\right) = (4.125, 5.375$

☆ Output

Optimal K Value = 3

Snehal B
10/4/26

Code:

```
import pandas as pd

import matplotlib.pyplot as plt

from sklearn.cluster import KMeans

from sklearn.preprocessing import StandardScaler


df = pd.read_csv("iris.csv")


X = df[['petal_length', 'petal_width']]


scaler = StandardScaler()

X_scaled = scaler.fit_transform(X)


sse = []

k_range = range(1, 11)


for k in k_range:

    km = KMeans(n_clusters=k, n_init='auto', random_state=42)

    km.fit(X_scaled)

    sse.append(km.inertia_)


plt.figure(figsize=(8, 5))

plt.plot(k_range, sse, marker='o', linestyle='-', color='b')

plt.title('Elbow Plot for Optimal K')
```

```
plt.xlabel('Number of Clusters (K)')
plt.ylabel('Sum of Squared Errors (SSE)')
plt.xticks(k_range)
plt.grid(True)
plt.show()

optimal_k = 3

km_final = KMeans(n_clusters=optimal_k, n_init='auto', random_state=42)
cluster_labels = km_final.fit_predict(X_scaled)

df['cluster'] = cluster_labels
plt.figure(figsize=(8, 5))

colors = ['red', 'green', 'blue']
for i in range(optimal_k):
    cluster_data = X_scaled[cluster_labels == i]
    plt.scatter(cluster_data[:, 0], cluster_data[:, 1],
                c=colors[i], label=f'Cluster {i}', alpha=0.7)

centroids = km_final.cluster_centers_
plt.scatter(centroids[:, 0], centroids[:, 1],
            c='black', marker='X', s=150, label='Centroids')
```

```
plt.title('Iris Flower Clusters (Petal Length vs Width)')

plt.xlabel('Scaled Petal Length')

plt.ylabel('Scaled Petal Width')

plt.legend()

plt.grid(True)

plt.show()


print("Clustering complete. Data points successfully assigned to clusters!")
```

Program 11

Implement Dimensionality reduction using Principal Component Analysis (PCA) method.

Screenshot:

DATE: 10/4/26 PAGE:

Lab - 10

Dimensionality Reduction using Principle Component Analysis (PCA)

Q PCA:

Mean of $X_1 = \mu_1 = \frac{4+8+13+7}{4} = 8$

Mean of $X_2 = \mu_2 = \frac{11+4+5+14}{4} = 8.5$

The points are : (4,11), (8,4), (13,5), (7,14)

Mean-centering the points, we get

(4 - 8, 11 - 8.5), (8 - 8, 4 - 8.5), (13 - 8, 5 - 8.5), (7 - 8, 14 - 8.5)

$\Rightarrow$ (-4, 2.5), (0, -4.5), (5, -3.5), (-1, 5.5)

Mean-centered data matrix Z is :

$$Z = \begin{bmatrix} -4 & 2.5 \\ 0 & -4.5 \\ 5 & -3.5 \\ -1 & 5.5 \end{bmatrix}$$

Since $\lambda_1 > \lambda_2$, the first principal component is

$$e_1 = \begin{bmatrix} 0.5574 \\ -0.8303 \end{bmatrix}$$

$P = Z \cdot e_1$

For each example,

$P_1 = (-4)(0.5574) + (2.5)(-0.8303) = -4.30535$

$P_2 = (0)(0.5574) + (-4.5)(-0.8303) = 3.73635$

$P_3 = (5)(0.5574) + (-3.5)(-0.8303) = 5.69305$

$P_4 = (-1)(0.5574) + (5.5)(-0.8303) = -5.12405$

Hence, the above are PC_1 values for all examples.

★ Output

--- Model Performance (Original) ---

Logistic Regression : 0.8533

Support Vector Machine (SVM) : 0.8750

Random Forest : 0.8750

Original number of features : 15

Number of features after PCA : 13

--- Model Performance (PCA) ---

Logistic Regression : 0.8533

Support Vector Machine (SVM) : 0.8804

Random Forest : 0.8641

★ Accuracy Comparison (Difference b/w Original vs PCA)

Logistic Regression :	+0.0000
SVM :	+0.0054
Random Forest :	−0.0109

★ ① Logistic regression is a linear model & PCA is a linear transformation, ∴ no change.

② The removal of the 2 dimensions likely stripped away noise or highly correlated features, leading to an increase in accuracy for the SVM model

③ Random Forest saw a slight decrease in accuracy because decision trees perform better on raw feature axes rather than the mixed components generated by PCA.

Code:

```
import pandas as pd

from sklearn.model_selection import train_test_split

from sklearn.preprocessing import LabelEncoder, StandardScaler

from sklearn.svm import SVC

from sklearn.linear_model import LogisticRegression

from sklearn.ensemble import RandomForestClassifier

from sklearn.metrics import accuracy_score

from sklearn.decomposition import PCA



df = pd.read_csv("heart.csv")


X = df.drop('HeartDisease', axis=1)

y = df['HeartDisease']


categorical_cols = X.select_dtypes(include=['object', 'category']).columns


X = pd.get_dummies(X, columns=categorical_cols, drop_first=True)


X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)


scaler = StandardScaler()

X_train_scaled = scaler.fit_transform(X_train)
```

```
X_test_scaled = scaler.transform(X_test)


def evaluate_models(X_train, X_test, y_train, y_test, scenario_name=""):
    print(f"--- Model Performance {scenario_name} ---")


    models = {
        "Logistic Regression": LogisticRegression(max_iter=1000, random_state=42),
        "Support Vector Machine (SVM)": SVC(random_state=42),
        "Random Forest": RandomForestClassifier(random_state=42)
    }


    accuracies = {}
    for name, model in models.items():
        model.fit(X_train, y_train)
        y_pred = model.predict(X_test)
        acc = accuracy_score(y_test, y_pred)
        accuracies[name] = acc
        print(f"{name}: {acc:.4f}")


    return accuracies


print("Evaluating models on Original Features (Scaled):")
original_accuracies = evaluate_models(X_train_scaled, X_test_scaled, y_train, y_test, "(Original)")
print("\n")
```

```
pca = PCA(n_components=0.95, random_state=42)

X_train_pca = pca.fit_transform(X_train_scaled)
X_test_pca = pca.transform(X_test_scaled)

print(f"Original number of features: {X_train_scaled.shape[1]}")
print(f"Number of features after PCA: {X_train_pca.shape[1]}\n")

print("Evaluating models on PCA-Reduced Features:")
pca_accuracies = evaluate_models(X_train_pca, X_test_pca, y_train, y_test, "(PCA)")
print("\n")

print("--- Accuracy Comparison (Original vs PCA) ---")
for model_name in original_accuracies.keys():
    orig_acc = original_accuracies[model_name]
    pca_acc = pca_accuracies[model_name]
    diff = pca_acc - orig_acc
    print(f"{model_name}:")
    print(f"  Original: {orig_acc:.4f} | PCA: {pca_acc:.4f} | Difference: {diff:+.4f}")
```